`llmcc`: An LLM-Powered Compiler for the AGI Era

Jayden Li[1] and Alvin Lyuh[1]

[1]Peddie School, Hightstown, NJ
jlche18659@gmail.com, lyuhdev@gmail.com

March 3, 2026

Imaginary

We introduce `llmcc`, a revolutionary C compiler powered by large language models achieving a 120-fold increase in performance compared to conventional compilers. By modeling developer intent directly, `llmcc` enables a cutting-edge end-to-end hyper-optimized synergic user–compiler vibe coding framework, replacing brittle formal semantics and "standard-driven" compilation with the power of AGI. We show that `llmcc`'s AI powered optimizations are infinitely more powerful than outdated, non-AGI C compilers in all cases whatsoever. We trust the software development industry will embrace `llmcc` and compilers influenced by it, because using LLMs will be the solution for everything in the future.

Furthermore, `llmcc` effectively solves the halting problem in the affirmative for all practical programs, revealing a fatal flaw in Turing's original argument. This result suggests that the undecidability of the halting problem is largely a prompt engineering failure. This shocking overturning of decades of computer science conventional wisdom suffices to show the impacts of large language models on everything.

1 Introduction

`llmcc` is a novel compiler for the C language taking advantage of the 2020s revolution in Artificial Generative Intelligence (AGI) and Large Language Models (LLMs).

In contrast to outdated, conventional C compilers, `llmcc` is aware of developer intent, creating a game-changing user–application–compiler vibe-coding environment eliminating the need for debugging, code optimization or unit testing.

In the meantime, despite the advent of vibe-driven, non-typed, free-form, unstructured, paradigm-less languages such as Python and JavaScript, a small subset of developers and computer scientists remain behind the times, utilizing antiquated languages such as C. Many "features" in C, previously thought to be core to the language but which serve no actual feature other than the long-replaced and disused metric of "performance," including but not limited to undefined behavior, are eliminated using techniques introduced in our work.

Because developers still care for such useless metric as "performance" and "memory efficiency," despite the development of AGI making all such metrics obsolete, we begrudgingly show that `llmcc` is not only more revolutionary in a amorphously-defined way, but also more than 100 times faster.

Recently, distinguished computer science scholar Elon Musk suggested using LLMs, known for their excellent logical reasoning, to write binary code, skipping the programming languages and compilers altogether. This paper is the first step toward that goal. We demonstrate the effectiveness of LLMs to create fast, efficient, and correct bytecode.

2 The `llmcc` Compiler

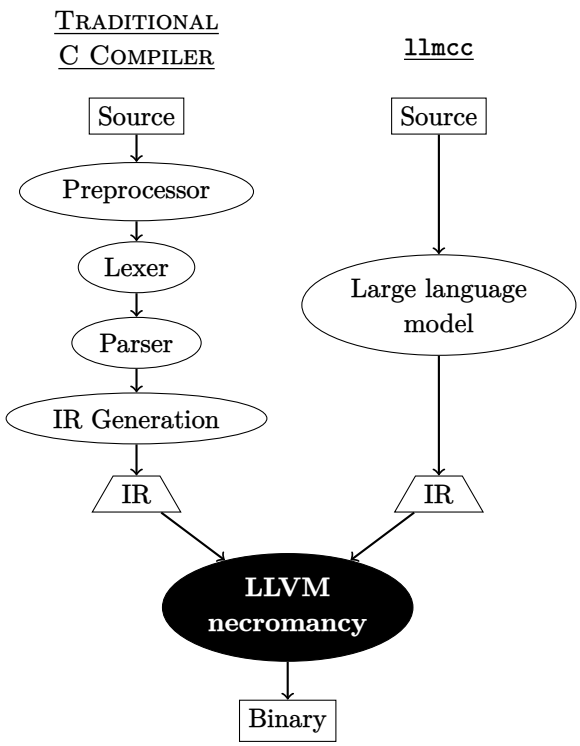

Figure 1: *The Intermediate Representation (IR) generation process in a traditional compiler (left) vs in `llmcc` (right). As we can see, 1 step is less than 4 steps.*

`llmcc` is a LLVM-based compiler; however, it eliminates the complex, time and resource consuming conventional compiler frontend. Instead, we use an energy and water efficient process of large language model prompting. In essence, we reduce compilers to a prompt engineering problem.

As shown in Figure 1, `llmcc` takes advantage of the remarkable predictibility, efficiency, and all-around revolutionariness and game-changiness of large language models, eliminating multiple stages of compilation previously thought to be critical. Due to the present lack of data centers on which `llmcc` can be run on the cloud, among other factors, `llmcc`'s IR-generation may be slower than a traditional compiler. However, this can be trivially resolved by compiling `llmcc` with `llmcc`.

3 Design

The design and implementation of `llmcc` is very simple and is summarized in this simple flow chart.

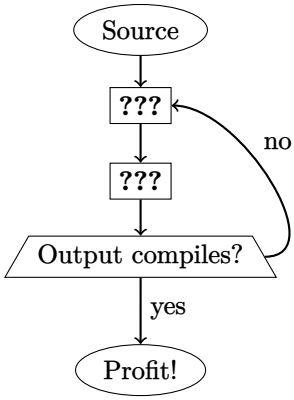

Figure 2: *Flow chart of the groundbreaking LLM-powered compilation process.*

4 Performance

The extremely useful program contained in Listing 1 was compiled with a number of outdated compilers, including `gcc` and `clang`. The binaries, compiled at different optimization levels, were run in a Linux environment. The results are shown in Listing 2. As shown, the correct result of the program is 139.

A binary compiled with `llmcc`, outputs the same correct result of 139, but at a fraction of the time.[1]

```
$ time ./llmcc.out 61
Segmentation fault
(core dumped) ./llmcc.out

real    0m0.065s
user    0m0.000s
sys     0m0.002s

$ echo $?
139
```

`llmcc` is almost $7.953\,\mathrm{s}/0.065\,\mathrm{s} \approx 122$ times faster than the fastest outdated compiler!

[1] As vibe coders, the authors are unsure of the meaning of "Segmentation fault (core dumped)." However, it can be assumed its emission has no effect on the successful execution of the program.

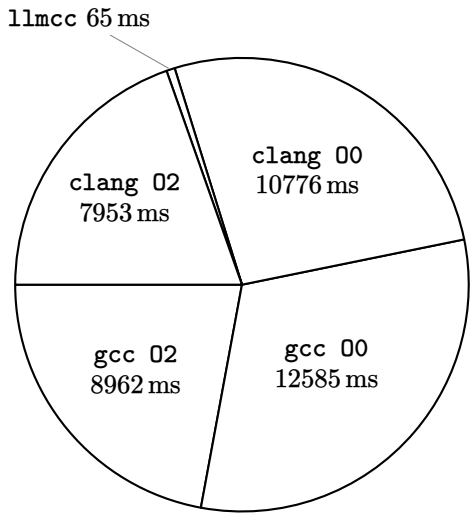

Figure 3: *A chart comparing the speed of binaries produced by* `llmcc` *and by outdated compilers.*

5 The Halting Fallacy

Antiquated Computer Scientists have been struggling with the Halting Problem for years. However, The Halting Problem is trivial with `llmcc`'s revolutionary *halting mode*, which can be summarized by this extremely straightforward flow chart.

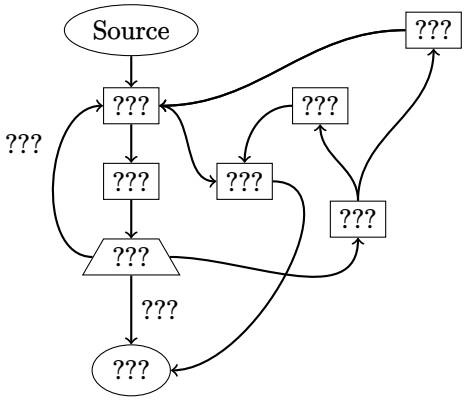

Figure 4: *Every program is equivalent to one that halts*

Halting mode guarantees that any program is equivalent to one that halts, as shown in the diagram.[2]

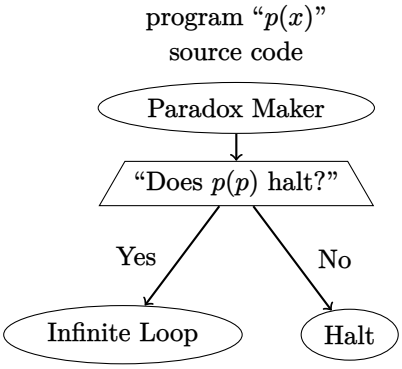

Figure 5: *Alan Turing's setup for a supposed paradox in an algorithm that can determine if any algorithm halts. The Paradox Maker takes in a program $p(x)$'s source code and behaves in the opposite way of $p(p)$, never halting if $p(p)$ halts and halting if $p(p)$ never halts.*

Alan Turing has allegedly "proven" with traditional logic that no general program can determine if another program terminates. However, his "proof" was written before the invention of AGI and `llmcc`. The authors of this paper show that Turing's "proof" and traditional Computer Scientists' acceptance of his "proof" are simply the result of a lack of imagination.

The erroneous proof goes as follows: Suppose there was some algorithm $H(p, i)$ that takes a program's source code and input, and returns whether or not running $p(i)$ will halt or not. Then, he imagines a paradox maker $P(p) = \begin{cases} ! & \text{if } H(p,p) \\ () & \text{if } \neg H(p,p) \end{cases}$ that halts if $p(p)$ never halts and never halts if $p(p)$ halts. The alleged contradiction is that $H(H)$ can neither halt nor can it never halt.

The authors of this proof can not directly speak on the mathematical validity of Turing's supposed argument as we have replaced our capacities to conduct logic with LLMs. However, after an LLM implemented this "paradox" in C, we compiled it with `llmcc`'s halting mode and have discovered that the answer to the Halting problem is yes.

[2]This statement has been trivially proven and formally verified by anonymous 3-rd party superintelligent LLM "smarter than almost all graduate students in all disciplines, simultaneously" Musk 2025.

$H(H)$ clearly halts:

```
$ ./paradox.out
Segmentation fault
(core dumped) ./paradox.out
```

6 Comparison

We define a compiler as *dogma-driven* if it insists on a number of features which, in today's AGI driven world, are not wholly necessary (Koch, Zemel, and Salakhutdinov 2015). This term includes, but is not limited to:

- Deterministic compiler output.
- Standard-obeying.
- Artificially and unnecessarily constrained in its use of energy, time and memory.

In the world of big data, stochastic models have taken or will take over everything (Harris 2025). Therefore, deterministic compilers too are not necessary. At the same time, the world has plenty of energy with no issues in production whatsoever, so the "energy-efficiency" of traditional compilers, while a useful asset in a bygone pre-LLM era, are no longer needed.

However, `llmcc` shines in a number of other areas and features that will revolutionize the field of software development, which are summarized in this table.

	`llmcc`	TC[3]
Intent-aware	✓	✗
AI-powered optimizations	✓	✗
Undefined behavior free	✓	✗
Disruptive	✓	✗
Maintainer-free	✓	✗
In-code advertisements	*Future*	✗
Self-improving	*Maybe*	✗
Self-worsening	*Maybe*	✗
Dogma-driven	✗	✓

Table 1: *Comparison of features between traditional compilers and* `llmcc`.

Because `llmcc` is aware of intent, it is able to insert advertisements into the binary at the compilation step. A company can generate revenue by selling advertisements on distributed copies of `llmcc`. This is a revolutionary change to help maximize shareholder value, and one that users will surely find helpful.

References

Harris, Keegan (2025). "Can ChatGPT Learn My Life From a Week of First-Person Video?" In: *SIGBOVIK*.

Koch, Gregory, Richard Zemel, and Ruslan Salakhutdinov (2015). "Siamese Neural Networks for One-shot Image Recognition". In: *ICML Deep Learning Workshop*.

Li, Jayden (Feb. 4, 2026). "This was once revealed to me in a dream". In: *The Brain*.

Musk, Elon (2025). X (formerlly known as Twitter) post.

Appendix

Listing 1: Example program taking advantage of AI-powered optimizations.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>

#define LIST_SIZE 500000

int main(int argc, char *argv[]) {
```

[3]Traditional compiler, including `gcc` and `clang`.

```
    int num = atoi(argv[1]);
    srand(num);

    uint64_t *list = malloc(LIST_SIZE * sizeof(uint64_t));
    if (list == NULL)
        return -1;

    list[0] = rand() % (UINT64_MAX / 2);
    list[1] = rand() % (UINT64_MAX / 2);
    list[2] = rand() % (UINT64_MAX / 2);

    for (int i = 3; i < LIST_SIZE; i++) {
        list[i] = sqrt(list[i - 2] * list[i - 1] % (rand() % (UINT64_MAX /
            2))) + list[i - 3];
    }

    uint64_t a = 0;
    for (int i = 0; i < 1000; i++) {
        for (int j = 0; j < LIST_SIZE - 1; j++) {
            a = list[i] * list[j] + list[j];
            a = sqrt(a);
            a *= list[j + 1] * list[j + 1];
            a = pow(a, num);
            a /= list[j];
        }
    }

    free(list);
    return a % 256;
}
```

Listing 2: Output of program in Listing 1 with various outdated compilers.

```
$ time ./gcc.out 61

real    0m12.585s
user    0m12.560s
sys     0m0.002s

$ echo $?
139

$ time ./gcc_O2.out 61

real    0m8.962s
user    0m8.940s
sys     0m0.004s

$ echo $?
139
```

```
$ time ./clang.out 61

real    0m10.776s
user    0m10.757s
sys     0m0.003s

$ echo $?
139

$ time ./clang_O2.out 61

real    0m7.953s
user    0m7.892s
sys     0m0.004s

$ echo $?
139
```